

Creazione e Gestione degli Oggetti Database (DDL)

Una guida pratica per creare, modificare e gestire oggetti database in Oracle: tabelle, viste, indici, sequenze, funzioni, procedure, package e trigger.

Obiettivi della lezione:

- Creare oggetti database con sintassi DDL corretta
- Modificare la struttura degli oggetti esistenti
- Gestire il ciclo di vita degli oggetti nel tempo

Prerequisiti: Conoscenza di SELECT, JOIN, DML (INSERT, UPDATE, DELETE) e funzioni SQL di base. Questa lezione si concentra esclusivamente sul linguaggio DDL (Data Definition Language) in ambiente Oracle Database.

DDL – Panoramica dei Comandi

Il **Data Definition Language (DDL)** è il sottoinsieme di SQL dedicato alla definizione e gestione della struttura del database. A differenza del DML (INSERT, UPDATE, DELETE), i comandi DDL operano sugli **oggetti schema** e non sui dati. Ogni comando DDL esegue un **COMMIT implicito**: le modifiche sono permanenti e non reversibili con ROLLBACK.

CREATE

Crea un nuovo oggetto database: tabella, vista, indice, sequenza, funzione, procedura, package o trigger.

ALTER

Modifica la struttura di un oggetto esistente senza eliminarlo. Aggiunge colonne, modifica vincoli o ricompila oggetti.

DROP

Elimina definitivamente un oggetto dal database. L'operazione è irreversibile e rimuove anche i dati associati.

TRUNCATE

Svuota completamente una tabella mantenendo la struttura. Molto più veloce di DELETE perché non genera undo log riga per riga.

Nota importante: I comandi DDL eseguono un COMMIT automatico prima e dopo l'esecuzione. Non è possibile fare ROLLBACK di un CREATE TABLE o di un ALTER TABLE.

CREATE TABLE – Definire la Struttura Dati

Concetti Chiave

- Ogni tabella è definita da **colonne** con nome e tipo di dato
- I **vincoli** garantiscono l'integrità dei dati
- **PRIMARY KEY**: identifica univocamente ogni riga
- **NOT NULL**: impedisce valori nulli su colonne obbligatorie
- **UNIQUE**: garantisce assenza di duplicati
- **FOREIGN KEY**: riferisce una chiave esterna
- **CHECK**: valida un range di valori accettabili

Esempio Pratico

```
CREATE TABLE employees_test (  
    employee_id  NUMBER(5)      PRIMARY KEY,  
    first_name   VARCHAR2(50)   NOT NULL,  
    last_name    VARCHAR2(50)   NOT NULL,  
    salary       NUMBER(8,2),  
    hire_date    DATE           DEFAULT SYSDATE,  
    department_id NUMBER(4)  
);
```

La clausola **DEFAULT** assegna un valore predefinito se non specificato. In Oracle i tipi **NUMBER** e **VARCHAR2** sono i più utilizzati. **DATE** include anche ora e minuti.

CREATE VIEW – Astrazione e Semplificazione

Una **vista** è una query SELECT salvata nel dizionario dati con un nome. Non contiene dati propri: restituisce dinamicamente i risultati della query sottostante ogni volta che viene interrogata. Le viste semplificano l'accesso ai dati, nascondono la complessità delle JOIN e possono essere usate per limitare la visibilità di alcune colonne.

Sintassi ed Esempio

```
CREATE VIEW v_emp_active AS
SELECT
  employee_id,
  first_name || ' ' || last_name AS full_name,
  salary,
  department_id
FROM employees
WHERE status = 'ACTIVE';
```

Utilizzo della vista:

```
SELECT full_name, salary
FROM v_emp_active
WHERE department_id = 50;
```

Vantaggi delle Viste

- Semplificazione**
Nascondono query complesse con JOIN e calcoli dietro un nome semplice.
- Sicurezza**
Espongono solo le colonne necessarie, proteggendo dati sensibili.
- Indipendenza logica**
Se la struttura delle tabelle cambia, basta modificare la vista senza impattare le applicazioni.

CREATE INDEX e CREATE SEQUENCE

CREATE INDEX – Ottimizzare le Performance

Un indice è una struttura ausiliaria che accelera la ricerca dei dati. Oracle crea automaticamente un indice sulla PRIMARY KEY, ma è utile crearne altri sulle colonne usate frequentemente nelle clausole WHERE, JOIN e ORDER BY.

```
-- Indice su cognome per ricerche frequenti
CREATE INDEX idx_emp_lastname
  ON employees(last_name);

-- Indice composto su più colonne
CREATE INDEX idx_emp_dept_salary
  ON employees(department_id, salary);
```

Attenzione: Gli indici migliorano le letture (SELECT) ma rallentano le scritture (INSERT, UPDATE, DELETE) perché devono essere aggiornati ad ogni modifica.

CREATE SEQUENCE – Generare ID Automatici

Una sequenza genera numeri progressivi in modo indipendente dalle tabelle. È lo strumento standard in Oracle per generare chiavi primarie numeriche. A partire da Oracle 12c è possibile usare anche **IDENTITY COLUMN**, ma le sequenze restano molto diffuse.

```
CREATE SEQUENCE seq_emp_id
  START WITH 1000      -- primo valore
  INCREMENT BY 1       -- passo
  MAXVALUE 999999     -- limite massimo
  NOCYCLE             -- non ricomincia da capo
  CACHE 20;           -- prealloca 20 valori
```

Utilizzo in INSERT: **VALUES (seq_emp_id.NEXTVAL, ...)**

CREATE FUNCTION – Logica Riutilizzabile

Caratteristiche

- Restituisce **sempre un valore** tramite la clausola RETURN
- Può essere chiamata in una SELECT, WHERE o espressione
- Accetta parametri di input (IN) e opzionalmente OUT/IN OUT
- Deve dichiarare il tipo del valore restituito
- Supporta **CREATE OR REPLACE** per sovrascrivere senza errori

📌 Le funzioni devono essere **deterministiche** (stesso input → stesso output) per essere usate in indici function-based.

Esempio Completo

```
CREATE OR REPLACE FUNCTION get_bonus(  
    p_salary IN NUMBER  
) RETURN NUMBER IS  
    v_bonus NUMBER;  
BEGIN  
    v_bonus := p_salary * 0.10;  
    RETURN v_bonus;  
EXCEPTION  
    WHEN OTHERS THEN  
        RETURN 0;  
END get_bonus;  
/
```

Utilizzo in una query:

```
SELECT  
    first_name,  
    salary,  
    get_bonus(salary) AS bonus  
FROM employees;
```

CREATE PROCEDURE – Esecuzione di Operazioni

Una **procedura** è un blocco PL/SQL memorizzato che esegue operazioni sul database. A differenza della funzione, **non restituisce un valore** direttamente, ma può modificare dati e restituire risultati tramite parametri OUT. È ideale per operazioni di aggiornamento, validazione complessa o batch.

Sintassi ed Esempio

```
CREATE OR REPLACE PROCEDURE update_salary (  
  p_emp_id IN      NUMBER,  
  p_percent IN     NUMBER DEFAULT 10,  
  p_new_sal OUT    NUMBER  
) IS  
BEGIN  
  UPDATE employees  
    SET salary = salary * (1 + p_percent/100)  
  WHERE employee_id = p_emp_id;  
  
  SELECT salary INTO p_new_sal  
  FROM employees  
  WHERE employee_id = p_emp_id;  
EXCEPTION  
  WHEN NO_DATA_FOUND THEN  
    RAISE_APPLICATION_ERROR(-20001, 'Dipendente non trovato');  
END update_salary;  
/
```

Confronto Function vs Procedure

Aspetto	Function	Procedure
Valore di ritorno	Obbligatorio (RETURN)	Opzionale (parametri OUT)
Uso in SELECT	Sì	No
Effetti collaterali	Limitati	Completi (DML)
Caso d'uso tipico	Calcoli, trasformazioni	Aggiornamenti, batch

Chiamata procedura:

```
DECLARE  
  v_new_sal NUMBER;  
BEGIN  
  update_salary(101, 15, v_new_sal);  
  DBMS_OUTPUT.PUT_LINE('Nuovo: ' || v_new_sal);  
END;  
/
```

CREATE PACKAGE – Organizzare il Codice

Un **package** è un contenitore logico che raggruppa funzioni, procedure, variabili e tipi correlati. Separa l'**interfaccia pubblica** (specification) dall'**implementazione** (body), permettendo di modificare il codice interno senza impattare chi usa il package. I package migliorano modularità, riutilizzo e performance (gli oggetti vengono caricati in memoria una sola volta).

Package Specification – Interfaccia

```
CREATE OR REPLACE PACKAGE pkg_emp AS
  -- Costante pubblica
  c_bonus_rate CONSTANT NUMBER := 0.10;

  -- Funzione pubblica
  FUNCTION get_bonus(p_salary NUMBER)
    RETURN NUMBER;

  -- Procedura pubblica
  PROCEDURE raise_salary(
    p_emp_id NUMBER,
    p_pct      NUMBER DEFAULT 5
  );
END pkg_emp;
/
```

Package Body – Implementazione

```
CREATE OR REPLACE PACKAGE BODY pkg_emp AS

  -- Variabile privata (non visibile fuori)
  v_log_count NUMBER := 0;

  FUNCTION get_bonus(p_salary NUMBER)
    RETURN NUMBER IS
  BEGIN
    RETURN p_salary * c_bonus_rate;
  END;

  PROCEDURE raise_salary(
    p_emp_id NUMBER,
    p_pct      NUMBER DEFAULT 5
  ) IS
  BEGIN
    UPDATE employees
      SET salary = salary * (1 + p_pct/100)
    WHERE employee_id = p_emp_id;
    v_log_count := v_log_count + 1;
  END;

END pkg_emp;
/
```



Vantaggio chiave: Modificando il PACKAGE BODY non è necessario ricompilare gli oggetti che lo chiamano, purché la SPECIFICATION resti invariata.

CREATE TRIGGER – Automazione degli Eventi

Un **trigger** è un blocco PL/SQL associato a una tabella che viene eseguito **automaticamente** da Oracle quando si verifica un evento specifico (INSERT, UPDATE, DELETE). I trigger sono fondamentali per garantire regole di business a livello di database, audit trail e validazioni complesse.



BEFORE

Eseguito **prima** dell'operazione. Utile per validare o modificare i dati in ingresso (es. impostare valori default).



AFTER

Eseguito **dopo** l'operazione. Utile per audit, log o propagazione di modifiche ad altre tabelle.



FOR EACH ROW

Trigger **row-level**: si esegue una volta per ogni riga interessata. Può accedere a :OLD e :NEW.



Statement-Level

Senza FOR EACH ROW: si esegue **una sola volta** per l'intera operazione, indipendentemente dalle righe.

```
CREATE OR REPLACE TRIGGER trg_emp_salary
  BEFORE INSERT ON employees
  FOR EACH ROW
BEGIN
  -- Imposta salary di default se NULL
  :NEW.salary := NVL(:NEW.salary, 1000);
  -- Imposta hire_date automatica
  :NEW.hire_date := NVL(:NEW.hire_date, SYSDATE);
END trg_emp_salary;
/
```

ALTER, DROP e TRUNCATE – Ciclo di Vita degli Oggetti

ALTER – Modifica Struttura

Modifica un oggetto esistente senza distruggerlo. Operazioni comuni:

- Aggiungere o eliminare colonne
- Modificare tipo o dimensione di una colonna
- Aggiungere o eliminare vincoli
- Rinominare oggetti
- Compilare o invalidare oggetti PL/SQL

```
-- Aggiunge colonna
ALTER TABLE employees
  ADD bonus NUMBER(8,2);

-- Modifica tipo colonna
ALTER TABLE employees
  MODIFY first_name VARCHAR2(100);

-- Aggiunge vincolo
ALTER TABLE employees
  ADD CONSTRAINT chk_salary
  CHECK (salary > 0);
```

DROP – Elimina Oggetto

Elimina definitivamente un oggetto dal database con tutti i dati associati. Operazione irreversibile.

```
-- Elimina tabella
DROP TABLE employees_test;

-- Elimina con cascade (chiavi esterne)
DROP TABLE departments CASCADE CONSTRAINTS;

-- Elimina vista
DROP VIEW v_emp_active;

-- Elimina funzione
DROP FUNCTION get_bonus;
```

 **Attenzione:** DROP non può essere annullato con ROLLBACK. Usare sempre con cautela in produzione.

TRUNCATE – Svuota Dati

Rimuove tutte le righe da una tabella mantenendo la struttura. Molto più veloce di DELETE perché non genera undo log riga per riga e resetta gli eventuali storage.

```
-- Svuota tabella (DDL, commit implicito)
TRUNCATE TABLE employees;

-- Con reset dello storage
TRUNCATE TABLE employees
  STORAGE (INITIAL 1M NEXT 1M);
```

CREATE

Definisce nuovi oggetti

DROP

Elimina oggetto

ALTER

Modifica struttura

TRUNCATE

Svuota dati